

Valizent / Al Jazeera CRM

Production readiness audit

RELEASE DECISION

NO-GO

19 findings / 3 High / 14 Medium / 2 Low

Report date: **October 5, 2026**. Execution evidence: October 4-5. **Decision: NO-GO for a new production release of the tested code.** No application source fixes, dependency changes, deployments or APK rebuilds were performed by this audit.

Complete report edition. Original content, evidence references, tables and screenshots retained. The original Markdown is also embedded as a PDF attachment.

Local file links retain their original destinations and may require access to the audit workspace. Referenced external files are not reproduced in full.

Contents

Executive assessment	4
Snapshot and limits of assurance	5
Architecture and attack surface	6
Verification results	8
Findings overview	9
Security	10
S01 - Worker accounts can read outsourced-worker PII and private documents	10
S02 - Forced password change is enforced only by the UI	11
Auth / Permission Gaps	12
A01 - A joint coordinator can list a deployment but cannot open it	12
Functional Bugs	13
F01 - Partial employee edits silently clear additional-document metadata	13
F02 - Settlement overrides can persist Infinity as financial amounts	14
F03 - Concurrent final-settlement creation stores duplicates	15
F04 - Leave-type PATCH permits a Sick policy that crashes submissions	16
F05 - Leave entitlement combines requests from different future years	17
F06 - A single calendar day can consume two leave days	18
F07 - Worker payslip page calls endpoints that do not exist	19
F08 - Literal site search can trigger a server error	20
F09 - Oversized JSON is reported as an internal server error	21
Performance / Reliability	22
P01 - Shared office IP is throttled before per-user limits matter	22
P02 - Target reporting repeats client-allocation queries per month	23

P03 - API requests have no application timeout	24
UI / Accessibility	25
U01 - Modal keyboard focus stays behind the dialog	25
Dead Code / Drift	26
O01 - Dashboard health message is static, not connected to health or alerts	26
D01 - Release automation has no quality gate, while both repositories fail lint	27
D02 - Small dead exports and duplicate initialization remain	28
Section Access, role drift and IDOR coverage	29
Secrets, injection, errors, dependencies and headers	30
Error tracking and operations	31
Performance and UX priorities	32
Dead code, duplicate logic and documentation handoff	33
Scores and production checklist	34
Remediation roadmap and release gate	35
Unverified items and decisions needed	36
Evidence and reproduction guide	37
Cleanup and visual evidence	38
Appendix - architecture diagram source	40

Executive assessment

The app has meaningful security controls and working operational flows, but it is not ready for an unconditional production handoff. Three confirmed High-severity blockers are outsourced-personnel disclosure to Worker accounts, accidental clearing of employee document metadata, and settlement numeric corruption. There are **19 confirmed findings: 0 Critical, 3 High, 14 Medium and 2 Low**. These are all quality findings, not 19 exploitable security vulnerabilities. The direct security findings are S01 (High) and S02 (Medium); A01 is an inconsistent authorization policy that blocks intended access.

The most useful speed work is batching target-report database queries, correcting shared-IP throttling, bounding API waits and paginating outsourced lists. Deleting small unused functions is housekeeping and is not supported as a cause of production slowness.

Your repositories are correctly recognized as **server/** and **client/**, each with real history (174 and 250 commits respectively at the inspected heads). Root history is archived and is not the active deployment repository. No Git setup or reinitialization is needed. Both repos have READMEs and architecture/history documentation; the issue is stale claims, not a lack of documentation.

Snapshot and limits of assurance

Server HEAD: c354b5a16d877beaab53a377c4f07bcd8c1d863f. Client HEAD: a9d429fe783e91bc27c53925f62c3ed78e173c84. See

[.qa-audit/production-2026-10-04/final-source-snapshot.json](#) for final uncommitted changes. Your edits to employee contract dates, EOSB simulations, attendance/layout and associated client forms were preserved. HTTP evidence was captured before some of those later edits; source checks confirm the reported employee default, final-settlement check/insert race and overflow paths remain. The whole latest dirty tree was not re-run as a fresh release candidate.

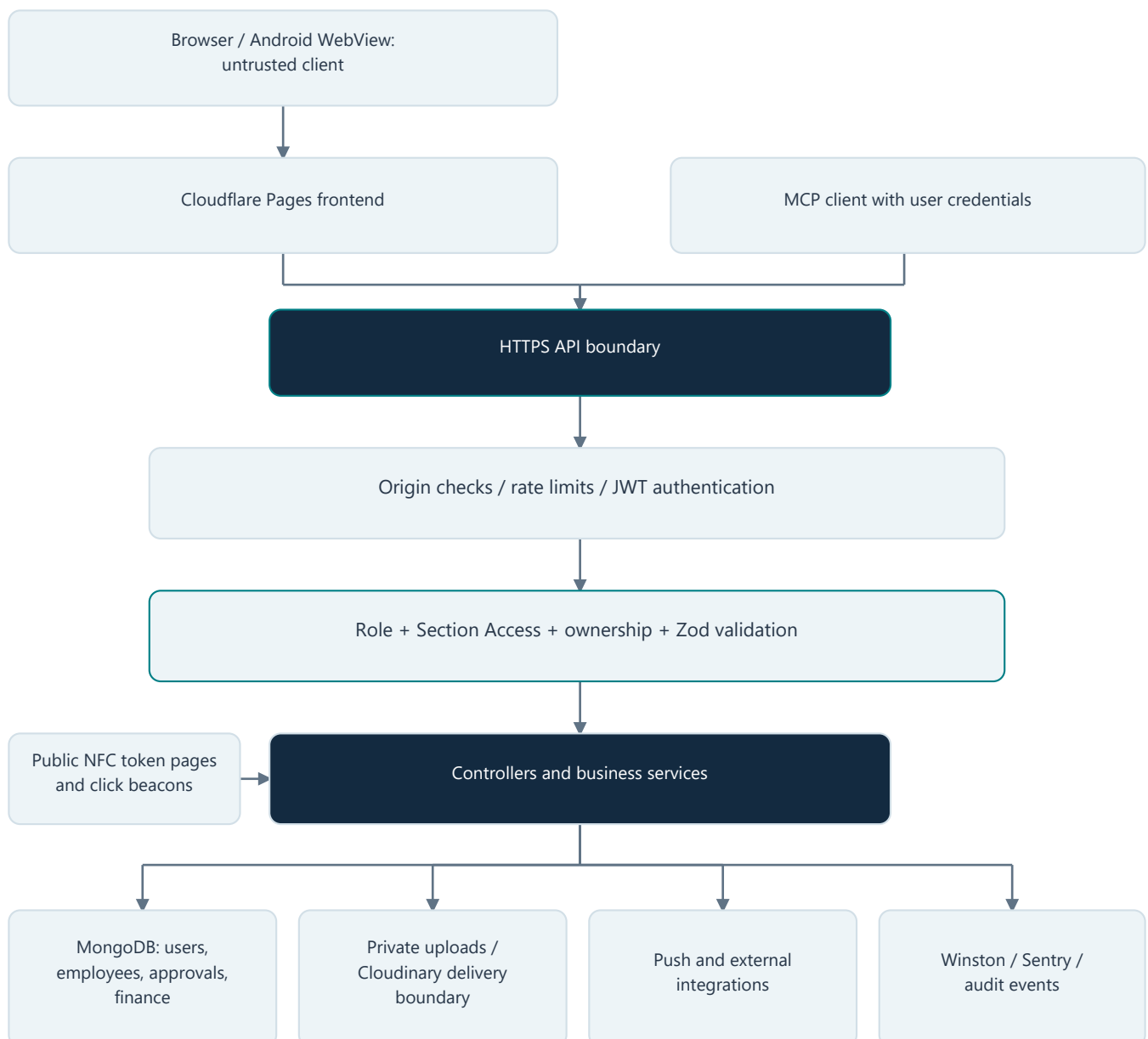
All mutation, role, concurrency and workload tests used an isolated local MongoDB replica set and synthetic accounts/data. External checks were restricted to authorized public responses. No real customer records were changed or downloaded. Local API ran on 5015 in production error mode and Vite on 5175, avoiding your existing ports. The API launcher used the real server entry point with isolated environment variables, rather than blindly loading the real database from npm run dev.

This is broad source/API coverage, **not a claim that every UI page, role-policy combination, database branch or infrastructure failure was exhaustively tested**. The route sweep uses empty bodies and nonexistent IDs and cannot prove happy-path ownership. A prior Worker browser loop captured loading states; those are explicitly excluded from page-pass claims. APK installation, full payroll generation-to-payslip, real email/push/storage delivery, live authenticated production behavior, backup restore and cloud-console configuration remain unverified. The payslip journey is currently blocked by F07.

Architecture and attack surface

This is an internal manpower-supply/trading CRM/ERP: employee records and documents feed mobilisation approvals, deployments, attendance and monthly hours; approved operational inputs feed invoicing, recorded client payments and allocations. Leave, timesheets, advances, EOSB and employee self-service operate alongside company settings, approval-role membership and section permissions. The NFC module separately serves public business-card pages.

Client: React 18, Vite 8, React Router 7, TanStack Query, Axios, React Hook Form/Zod, Tailwind and i18next; Capacitor wraps the frontend for mobile. Backend: Node ESM, Express 4, Mongoose 8/MongoDB, Zod 4, JWT/bcrypt, Multer/Cloudinary, PDFKit/ExcelJS/SheetJS, web-push, Winston and Sentry. A separate MCP service provides read tools over the API. Installed versions/configuration come from package files and lockfiles, not architecture prose alone.



Architecture map rendered from the source. Exact diagram source is retained in the appendix.

Authorization is layered: requireAuth identifies a current active user and checks token version; role floors still exist; Section Access grants come from approval-role membership; employee/team and record-owner checks must remain in services. A client hide is a UX aid, not an authorization boundary. Admin behavior and legacy role-specific actions should be documented rather than replaced indiscriminately with section permissions.

The authorized development frontend's deployed bundle points to **<https://tech-jazeera-backend.onrender.com/api>**. Its health endpoint returned 200 and empty login returned schema-level 400 with the correct allowed origin. The separately authorized nfc.valizent.com API also returned healthy; its rejection of the development frontend origin is not a finding because that is not the frontend's configured API. Server GitHub workflow describes Oracle/PM2 deployment, while development uses Render: environment-specific release mapping needs documentation. Actual database hosting/privileges, reverse-proxy settings, backups and deployed commit identity require operator access.

Verification results

Check	Result and evidence
Local boot	API health 200; Vite 200; Admin login/dashboard and Employees list rendered. Production-mode errors were generic.
Existing backend tests	34/34 passed, 8 files; .qa-audit/production-2026-10-04/server-tests-retry.log . An earlier parallel run had Mongo startup timeout; retry passed.
Frontend production build	Passed; .qa-audit/production-2026-10-04/client-build.log . Built into audit output, not the app's existing dist.
Lint	Server 1 error; client 30 errors / 20 warnings. See D01; not a clean quality gate.
Source parse	554 JS/JSEX source files parsed; no parse errors in the saved static scan.
Route coverage	294 declarations, 39 module route files plus app-level routes, all mounted. .qa-audit/production-2026-10-04/API-INVENTORY.md , .qa-audit/production-2026-10-04/API-TRACE.json .
Role boundary smoke	2,836 local requests across anonymous and nine role types; no 5xx in this basic sweep. Targeted valid-data tests found the failures below. .qa-audit/production-2026-10-04/role-sweep.json .
Authentication	Real login, refresh rotation, logout invalidating the old refresh token, secure HttpOnly cookie attributes, rejected untrusted origin and rejected object-valued login inputs. .qa-audit/production-2026-10-04/http-evidence.json .
Ownership	Coordinator own employee 200; foreign employee/document 403; foreign leave decision/cancel denied; foreign notification update 404; requirement/daily-update foreign access denied. .qa-audit/production-2026-10-04/workflow-evidence.json , .qa-audit/production-2026-10-04/scoping-performance.json .
Core workflow	Employee fixture → mobilisation create/submit/approve → deployment → monthly hours approve → invoice marker → client payment record/approve/allocation succeeded. An invoice before period end was rejected. .qa-audit/production-2026-10-04/billing-flow.log . This was a recorded local workflow, not an actual payment/email.
Leave / timesheets	Worker leave submit and authorized approval succeeded; timesheet submission and Admin approval succeeded; duplicate week rejected. Coordinator timesheet approval retains a legacy role restriction.
Hours / concurrency	900 hours for a 30-day month rejected by service cap; duplicate same-month entry produced one success and one conflict. Settlement concurrency failed separately.
Dates	Attendance leap-February, year boundary and Saturday-to-Friday week checks passed. Leave timestamp/year allocation checks failed separately.
Files	Fake PDF bytes with application/pdf rejected 400; encoded NFC traversal probe returned 404. Worker outsourced-file authorization failed with CDN stub, S01.
Browser	Admin dashboard, target modal, workforce hub and Employees list; Worker profile, payslip error and exit-document form inspected. Mobile 390px exit form fit without demonstrated overflow. Tablet/large-screen and every page are not fully certified.
Browser errors	Earlier dev-session AuthProvider/HMR errors were present in historical console entries; login/dashboard/Employees subsequently rendered. Do not interpret this as a proven production crash or claim the whole session had zero console errors.

Findings overview

ID	Severity	Finding
S01	High	Worker accounts can read outsourced-worker PII and private documents
S02	Medium	Forced password change is enforced only by the UI
A01	Medium	A joint coordinator can list a deployment but cannot open it
F01	High	Partial employee edits silently clear additional-document metadata
F02	High	Settlement overrides can persist Infinity as financial amounts
F03	Medium	Concurrent final-settlement creation stores duplicates
F04	Medium	Leave-type PATCH permits a Sick policy that crashes submissions
F05	Medium	Leave entitlement combines requests from different future years
F06	Medium	A single calendar day can consume two leave days
F07	Medium	Worker payslip page calls endpoints that do not exist
F08	Medium	Literal site search can trigger a server error
F09	Low	Oversized JSON is reported as an internal server error
P01	Medium	Shared office IP is throttled before per-user limits matter
P02	Medium	Target reporting repeats client-allocation queries per month
P03	Medium	API requests have no application timeout
O01	Medium	Dashboard health message is static, not connected to health or alerts
D01	Medium	Release automation has no quality gate, while both repositories fail lint
U01	Medium	Modal keyboard focus stays behind the dialog
D02	Low	Small dead exports and duplicate initialization remain

Security

S01 - Worker accounts can read outsourced-worker PII and private documents

ID: S01

Severity: High

Category: Security

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/employees/outsourcedEmployee.routes.js:29](#);
[server/src/modules/employees/outsourcedEmployee.service.js:17](#)

Component / API / data: GET /api/outsourced-employees; GET /api/outsourced-employees/:id; GET /api/outsourced-employees/:id/documents/:fileId/file; OutsourcedEmployee

Evidence: [.qa-audit/production-2026-10-04/http-evidence.json](#);
[.qa-audit/production-2026-10-04/deep-evidence.json](#). Worker list/detail returned 200. File route returned 200 and synthetic private bytes through the real auth/controller/service chain; only the external CDN fetch was stubbed.

What is happening / root cause: The client hides this section using employeeCreate, but these backend reads have authentication only. Neither a read permission nor record visibility is applied. The raw list limit also accepts 1,000,000 and returned all 1,201 synthetic records.

Why it matters / impact: Security: high confidentiality impact. Business: personnel/identity data exposure. Users: all outsourced workers in this collection. Likelihood: high with any Worker account. Exploitability: direct API request, no elevated role. Production: release blocker if this code is deployed.

How to reproduce: Log in as the disposable Worker. GET the collection and a seeded record ID, then its document file endpoint. Run http-tests.mjs and deep-tests.mjs against the isolated runtime. Do not repeat against real personnel records.

Recommended fix / architecture: Apply an explicit approved read policy to list, detail and file routes, including any required team/client scope. Reuse that policy in the client guard. Add a Zod query schema with a small maximum page size and project only needed list fields. Keep file authorization before signing/fetching.

Priority: P0 - before release.

Regression test required / verification: YES - Worker denied on all three paths; allowed role succeeds; forbidden coordinator denied; invalid/excessive list limits rejected; file bytes inaccessible through both record and download URLs.

S02 - Forced password change is enforced only by the UI

ID: S02

Severity: Medium

Category: Security

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/middleware/auth.js:34](#)

Component / API / data: requireAuth; authenticated API endpoints; User

Evidence: [.qa-audit/production-2026-10-04/latest-regressions.json](#): weak-password login returned forcePasswordChange:true; its bearer token still fetched /api/me/profile with 200.

What is happening / root cause: Authentication loads the user but does not restrict a forcePasswordChange session. Closing/bypassing the frontend flow is unnecessary: the API token already works.

Why it matters / impact: Security: password-remediation control bypass. Business/users: account remains usable with a credential the system says must change. Likelihood: medium, conditional on a flagged account. Exploitability: direct authenticated request; this does not grant a higher role.

How to reproduce: Use a disposable flagged/weak-password account, POST /api/auth/login, then GET /api/me/profile using the returned token without changing its password. See latest-regressions.mjs.

Recommended fix / architecture: Issue a restricted password-change session or reject normal protected endpoints while the flag is set. Explicitly allow password change and logout, and return a stable machine-readable code that the UI handles.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - flagged token cannot access normal resources; password change clears the restriction; logout still works; ordinary users remain unaffected.

Auth / Permission Gaps

A01 - A joint coordinator can list a deployment but cannot open it

ID: A01

Severity: Medium

Category: Auth / Permission Gaps

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/deployments/deployment.service.js:62;](#)
[server/src/modules/deployments/deployment.service.js:1490](#)

Component / API / data: GET /api/deployments and GET /api/deployments/:id

Evidence: [.qa-audit/production-2026-10-04/final-probes.json](#): confirmed joint CoordinatorB saw the deployment in a 200 list response, then received 403 on detail.

What is happening / root cause: List visibility follows mobilisation coordinators; detail visibility for an Employee worker follows the employee current coordinator. These are different policies for the same deployment.

Why it matters / impact: Security: no new unauthorized disclosure demonstrated beyond the intended list membership. Business/users: legitimate joint coordinator blocked. Likelihood: high for shared mobilisations. Exploitability: normal navigation. Production: workflow interruption.

How to reproduce: Create a mobilisation for CoordinatorA's worker; add CoordinatorB as a confirmed joint coordinator; grant needed section access. List as B, then request the returned deployment ID. See [final-probes.mjs](#).

Recommended fix / architecture: Define deployment visibility once using the intended mobilisation/worker policy and call it from list, detail, hours, invoices and downloads. Preserve stricter mutation permissions separately.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - both confirmed participants can perform intended reads; unrelated coordinator is excluded from list and detail; stale/removed membership is rejected.

Functional Bugs

F01 - Partial employee edits silently clear additional-document metadata

ID: F01

Severity: High

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/employees/employee.validation.js:94;](#)
[server/src/modules/employees/employee.validation.js:187](#)

Component / API / data: PATCH /api/employees/:id; Employee.additionalDocuments

Evidence: [.qa-audit/production-2026-10-04/latest-regressions.json](#): updating only department returned additionalDocuments: []. Rechecked the latest on-disk schema on October 5: `parse({department:'Audit'})` also emitted additionalDocuments: [].

What is happening / root cause: The shared schema applies `default([])` to additionalDocuments. Deriving the PATCH schema with `partial()` still materializes that default under the installed Zod version. The service writes the resulting empty array.

Why it matters / impact: Security: integrity rather than access-control issue. Business: document-register metadata loss. Users: any employee edited through a partial payload. Likelihood: high in normal partial updates. Exploitability: ordinary authorized PATCH. Production: release blocker; binary file deletion was not demonstrated.

How to reproduce: Seed an employee with one additional document. PATCH only `{"department":"QA department"}`. Read the employee and compare the array. [latest-regressions.mjs](#) records the HTTP proof.

Recommended fix / architecture: Remove create defaults from the shared PATCH object. Apply defaults only in create-specific schema/service and make updates preserve omitted keys. Review sibling optional defaults before changing them.

Priority: P0 - before release.

Regression test required / verification: YES - omitted documents survive PATCH; explicit `[]` clears only when requested; creation still gets the intended default; nested omitted fields are retained.

F02 - Settlement overrides can persist Infinity as financial amounts

ID: F02

Severity: High

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/eosb/settlement.validation.js:19](#);
[server/src/modules/eosb/settlement.service.js:24](#)

Component / API / data: POST /api/eosb; Settlement monetary fields

Evidence: [.qa-audit/production-2026-10-04/latest-regressions.json](#): overrideEosbGross:1e308 returned 201 with null monetary values in JSON; direct database check showed non-finite eosbGross and totalSettlement.

What is happening / root cause: An input may be finite while rounding by multiplying it by 100 overflows. The override has a minimum but no business maximum; computed values are persisted without a finite-value invariant.

Why it matters / impact: Security: integrity risk to an authorized financial workflow. Business: corrupted settlement totals/PDF/report inputs. Users: settlement operators and employees. Likelihood: low accidentally, easy with a crafted authorized request. Production: release blocker.

How to reproduce: POST /api/eosb for a disposable eligible employee with exitDate:2026-09-30, exitReason:EndOfContract, overrideEosbGross:1e308. Inspect both response and stored finite checks. See latest-regressions.mjs.

Recommended fix / architecture: Set realistic monetary/day bounds; validate every computed amount with Number.isFinite before persistence; use a consistent decimal or bounded minor-unit representation. Reject invalid results with 400/422 rather than storing them.

Priority: P0 - before release.

Regression test required / verification: YES - extreme finite values are rejected with no write; boundaries and normal rounding pass; all saved settlement amounts are finite.

F03 - Concurrent final-settlement creation stores duplicates

ID: F03

Severity: Medium

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/eosb/settlement.service.js:100;](#)
[server/src/modules/eosb/settlement.service.js:139](#)

Component / API / data: POST /api/eosb; Settlement

Evidence: [.qa-audit/production-2026-10-04/latest-regressions.json](#): five concurrent EndOfContract requests returned five 201 responses; five settlements existed for one employee.

What is happening / root cause: The existing-record check and insertion are separate operations without a database uniqueness invariant. Latest edits exclude CurrentEmployee simulations from the lookup, but do not remove the final-settlement race.

Why it matters / impact: Security: data-integrity issue for authorized callers. Business: duplicate liabilities/review work, not proof of duplicate cash payout. Users: HR/accounts. Likelihood: medium with retries/double-submit. Exploitability: simultaneous authorized requests.

How to reproduce: Send five parallel identical final-settlement requests for one fresh employee. Count stored final settlements. See [latest-regressions.mjs](#).

Recommended fix / architecture: Agree the final-settlement identity (employee plus employment episode if rehiring is supported). Enforce it with a suitable partial unique index that excludes CurrentEmployee simulations, plus idempotency and 409 handling. Reconcile existing duplicates before adding the index.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - concurrency stores exactly one final settlement; retried idempotency key returns the same result; simulations remain allowed; distinct employment episodes follow policy.

F04 - Leave-type PATCH permits a Sick policy that crashes submissions

ID: F04

Severity: Medium

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/leave/leave.validation.js:71](#);
[server/src/modules/leave/leave.service.js:149](#)

Component / API / data: PATCH /api/leave-types/:id; POST /api/me/leave; LeaveType

Evidence: [.qa-audit/production-2026-10-04/workflow-evidence.json](#): changing Manual to Sick without tiers returned 200; a subsequent Worker request returned 500.

What is happening / root cause: Create validation requires sickPayTiers, but update validation does not validate the merged policy's cross-field invariant. Submission assumes tiers are present.

Why it matters / impact: Security: no access bypass. Business: broken policy prevents leave requests. Users: employees assigned that type. Likelihood: medium during policy edits. Exploitability: authorized administrator misconfiguration.

How to reproduce: Create Manual leave type; PATCH {"recurrence":"Sick"}; submit leave using its ID. See workflows.mjs.

Recommended fix / architecture: Merge stored policy with the patch, validate the complete policy using shared refinements, then persist. Reject invalid transitions and protect calculation code from malformed historic policies.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - invalid Manual→Sick transition rejected; valid tiers accepted; existing valid Sick requests retain expected pay allocation.

F05 - Leave entitlement combines requests from different future years

ID: F05

Severity: Medium

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/leave/leave.service.js:89](#)

Component / API / data: POST /api/me/leave; annual eligibility / LeaveRequest aggregation

Evidence: [.qa-audit/production-2026-10-04/deep-evidence.json](#): a two-day annual allowance auto-approved June 2027; June 2028 then saw usedDays:2 and remainingDays:0 from the same 2026-based window.

What is happening / root cause: Eligibility uses the current service-year start and a lower-bound-only date filter. Future requests from different entitlement periods are pooled together.

Why it matters / impact: Security: entitlement integrity. Business/users: wrong automatic approval/review decisions and balances. Likelihood: medium when advance leave is allowed. Exploitability: ordinary request dates. Production: incorrect HR workflow.

How to reproduce: Create Annual policy with daysPerYear:2. Submit two days in June 2027, then two days in June 2028 for the same employee. See deep-tests.mjs.

Recommended fix / architecture: Define the entitlement period relative to the requested leave and bound both ends. Specify handling of cross-period leave and future booking limits. Preserve decision snapshots, while recomputing a fresh eligibility assessment for new requests.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - requests in separate service years do not consume each other; same-period requests do; anniversary and cross-period requests follow explicit policy.

F06 - A single calendar day can consume two leave days

ID: F06

Severity: Medium

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/leave/leave.service.js:45](#)

Component / API / data: POST /api/me/leave; inclusiveDays

Evidence: [.qa-audit/production-2026-10-04/input-probes.json](#): start 2026-10-25T06:00Z and end 2026-10-25T20:30Z returned days:2. Both timestamps fall on October 25 in UTC and Riyadh.

What is happening / root cause: Schemas accept timestamps but duration rounds elapsed fractional days before adding one. Calendar-day leave and elapsed-time arithmetic are mixed.

Why it matters / impact: Security: entitlement integrity. Business: incorrect leave balances. Users: API/mobile clients providing timestamps. Likelihood: medium. Exploitability: ordinary accepted input, no extra privilege.

How to reproduce: Submit the two timestamps above for a Manual leave type. Inspect days. See [input-probes.mjs](#).

Recommended fix / architecture: Require date-only inputs or normalize both dates to the chosen business calendar before inclusive day counting. Use one documented date convention across API and UI.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - same-day times always count one; adjacent calendar dates count two; leap day, DST-independent business timezone and year boundaries pass.

F07 - Worker payslip page calls endpoints that do not exist

ID: F07

Severity: Medium

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [client/src/features/ess/ess.api.js:194](#);

[client/src/features/ess/pages/MyPayslipsPage.jsx:53](#)

Component / API / data: GET /api/me/payslips; GET /api/me/payslips/:runId/pdf

Evidence: [.qa-audit/production-2026-10-04/deep-evidence.json](#);

[.qa-audit/production-2026-10-04/payslips-browser.png](#). Authenticated list request returned 404; UI displayed "Could not load your payslips" and a misleading connection/retry instruction. No matching route exists in the inventory.

What is happening / root cause: The client still presents payslip self-service without a mounted backend implementation. Retrying cannot resolve a missing endpoint.

Why it matters / impact: Security: no cross-user payslip access demonstrated. Business/users: payroll self-service unavailable. Likelihood: every visit. Exploitability: ordinary navigation. Production: feature is not ready.

How to reproduce: Sign in as Worker, open My payslips, or GET /api/me/payslips with its token. See deep-tests.mjs and screenshot.

Recommended fix / architecture: Either implement a scoped published-payslip API/PDF route using req.user.employee, or remove the unavailable feature until ready. Distinguish unsupported/404 from network failure in UI.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - employee sees only own finalized/published payslips; arbitrary run ID cannot download another employee's PDF; empty state and actual network failures differ.

F08 - Literal site search can trigger a server error

ID: F08

Severity: Medium

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/deployments/deployment.service.js:1490](#)

Component / API / data: GET /api/deployments?site=%5B

Evidence: [.qa-audit/production-2026-10-04/final-probes.json](#): site=[returned 500 with the generic production error.

What is happening / root cause: The site filter is passed directly into MongoDB \$regex. An unclosed bracket is treated as invalid regular-expression syntax instead of search text.

Why it matters / impact: Security: input-to-query robustness issue; catastrophic regex or denial of service was not tested or established. Business/users: legitimate searches fail. Likelihood: medium for punctuation searches. Exploitability: authenticated reader.

How to reproduce: GET /api/deployments?site=%5B as an allowed user. See final-probes.mjs.

Recommended fix / architecture: Use the existing escapeRegex utility for literal search. Retain query length bounds. If regex searching is actually required, make that an explicit bounded feature with safe validation.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - [, (, backslash and .* are literal search text; no 500; normal search and coordinator scoping remain correct.

F09 - Oversized JSON is reported as an internal server error

ID: F09

Severity: Low

Category: Functional Bugs

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/middleware/errorHandler.js:53](#)

Component / API / data: POST /api/auth/login with JSON exceeding the configured body limit

Evidence: [.qa-audit/production-2026-10-04/input-probes.json](#): an oversized body returned generic 500 rather than 413.

What is happening / root cause: normalizeError recognizes malformed JSON but not body-parser entity.too.large. A controlled client error becomes an operational alarm.

Why it matters / impact: Security: size limit still works; no payload acceptance or stack leak shown. Business: misleading alerts. Users: incorrect retry guidance. Likelihood: low for ordinary login. Exploitability: simple oversized request.

How to reproduce: Send a JSON body over 1 MB to local login; inspect status. See input-probes.mjs.

Recommended fix / architecture: Map entity.too.large to a safe 413 response and leave unknown errors masked. Apply analogous explicit mappings only where supported by middleware contracts.

Priority: P2 - scheduled cleanup.

Regression test required / verification: YES - large body gives 413 without stack; malformed JSON still gives 400; genuine bugs remain 500.

Performance / Reliability

P01 - Shared office IP is throttled before per-user limits matter

ID: P01

Severity: Medium

Category: Performance / Reliability

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/middleware/rateLimiter.js:30](#); [server/src/middleware/rateLimiter.js:49](#)

Component / API / data: Global API limiter and requireAuth user limiter

Evidence: [.qa-audit/production-2026-10-04/workflow-evidence.json](#): 602 requests from one synthetic IP alternated three accounts; request 601 was first 429, with approximately 200 requests per account.

What is happening / root cause: The global 600/15-minute IP budget remains below a single user's 1,200 budget. Adding a user limiter did not solve shared-office contention.

Why it matters / impact: Security: protective limiter exists. Business/users: ordinary staff share throttling and see failed loads. Likelihood: medium to high in shared offices with polling/tabs. Exploitability: no malicious actor needed. Production: intermittent apparent slowness.

How to reproduce: Run the isolated rate-budget loop in workflows.mjs; inspect first429:601. This traffic was local only.

Recommended fix / architecture: Keep strict unauthenticated/login limits; size the authenticated IP backstop for shared offices and enforce meaningful per-user budgets. Validate trusted proxy configuration before relying on forwarded IPs. Use shared limiter storage for multi-instance hosting.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - representative office traffic across several users stays below the shared ceiling; a single excessive user is throttled; spoofed forwarding cannot bypass the intended deployment boundary.

P02 - Target reporting repeats client-allocation queries per month

ID: P02

Severity: Medium

Category: Performance / Reliability

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/src/modules/mobilisationTargets/mobilisationTarget.service.js:98](#)

Component / API / data: Semi-annual target reporting; getClientAllocation; deployments/mobilisations/clientpayments

Evidence: [.qa-audit/production-2026-10-04/final-probes.json](#): warm six-month report, 12 clients: 234 database calls in 316 ms locally (78 deployments.find, 78 mobilisations.find, 72 clientpayments.find, six target reads).

What is happening / root cause: Each month independently computes client payment allocations, including repeated reads of the same client history. Work scales with months times clients.

Why it matters / impact: Security: none demonstrated. Business/users: report latency and database load grow with clients/history. Likelihood: high as usage grows. Exploitability: ordinary report. Production: measured query amplification, not a claimed production response time.

How to reproduce: Run scoping-performance.mjs to seed the 12-client fixture, then final-probes.mjs for the warm query count. The initial cold count included model initialization and is not used here.

Recommended fix / architecture: Load the required clients and financial histories once per request, compute allocations once per client, and reuse across month buckets. Then measure query count and p50/p95 on realistic sanitized data. Avoid a long-lived financial cache until invalidation is defined.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - totals match existing results for partial/approved payments and date boundaries; query count no longer multiplies full client-history reads by six.

P03 - API requests have no application timeout

ID: P03

Severity: Medium

Category: Performance / Reliability

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [client/src/lib/axios.js:48](#)

Component / API / data: Shared Axios client; session refresh and page requests

Evidence: [.qa-audit/production-2026-10-04/deep-evidence.json](#): source-identical Axios configuration had `timeout:0` and remained pending after 1.2 seconds against a deliberately hanging local HTTP server; the test explicitly canceled it.

What is happening / root cause: No finite request timeout bounds an unresponsive connection. Boot refresh and loading states depend on requests completing.

Why it matters / impact: Security: none established. Business/users: indefinite loading when a connection remains open but never responds. Likelihood: medium during infrastructure failures. Exploitability: not required. Production: failure-mode weakness; 1.2 seconds is a test observation, not proof that every slow request hangs forever.

How to reproduce: Run `deep-tests.mjs` hanging-server probe; inspect timeout and pending state. Only Vite constant/base URL setup was adapted for Node.

Recommended fix / architecture: Set endpoint-appropriate timeouts and cancellation; bound refresh separately; provide retryable errors. Do not automatically retry financial writes without idempotency.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - hanging refresh exits loading with a recoverable state; slow read times out; canceled/unmounted requests do not update UI; timed-out mutation is not blindly repeated.

UI / Accessibility

U01 - Modal keyboard focus stays behind the dialog

ID: U01

Severity: Medium

Category: UI / Accessibility

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [client/src/components/ui/Modal.jsx:42](#)

Component / API / data: Shared Modal; dashboard Manage targets

Evidence: [.qa-audit/production-2026-10-04/modal-focus.json](#);

[.qa-audit/production-2026-10-04/modal-focus.png](#). Reproduced October 5: opening Manage targets left focus on the opener; Tab moved to background Quick actions; insideDialog:false.

What is happening / root cause: The modal locks scrolling and handles Escape, but does not move focus into the dialog, contain it, or make the background inert.

Why it matters / impact: Security: none demonstrated. Business/users: keyboard and assistive-technology users can operate obscured background controls. Likelihood: every keyboard-opened modal. Exploitability: ordinary interaction. Production: accessibility failure.

How to reproduce: Sign in as Admin, activate Manage targets, press Tab and inspect `document.activeElement`. See `modal-focus.json`.

Recommended fix / architecture: Use an accessible dialog primitive or implement initial focus, focus containment, background inertness and focus restoration. Preserve close-disabled behavior for mandatory flows.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - Tab/Shift+Tab stay inside; Escape respects closable; closing restores opener focus; dialog has an accessible name.

Dead Code / Drift

001 - Dashboard health message is static, not connected to health or alerts

ID: O01

Severity: Medium

Category: Dead Code / Drift

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [client/src/features/dashboard/components/SystemLogsWidget.jsx:16](#)

Component / API / data: System & Software Health dashboard widget

Evidence: [.qa-audit/production-2026-10-04/modal-focus.png](#); source renders unconditional translation text. Browser showed "All critical system services are running normally. No active software alerts detected." after reproduced API 500s; error-log link was /# in the isolated environment.

What is happening / root cause: The component receives recentActivity, not service health or Sentry alert status. The wording claims live operational knowledge the component does not have.

Why it matters / impact: Security: no exploit demonstrated. Business: false confidence during failures. Users: administrators may miss incidents. Likelihood: every dashboard visit. Exploitability: not applicable. Production: misleading monitoring UX.

How to reproduce: Open Admin dashboard, inspect the health card and component. Induce a local failing site search; the card remains unchanged.

Recommended fix / architecture: Either label this as a static support/monitoring link or display an actual timed health result with last-checked and unknown/error states. Hide unavailable Sentry links instead of using #.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - absent monitoring configuration shows unknown/unavailable; failed health check never shows healthy; stale observations are labeled.

D01 - Release automation has no quality gate, while both repositories fail lint

ID: D01

Severity: Medium

Category: Dead Code / Drift

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [server/.github/workflows/deploy.yml:21](#);
[server/src/modules/mobilisationTargets/mobilisationTarget.service.js:196](#)

Component / API / data: Server GitHub deployment; server/client lint

Evidence: [.qa-audit/production-2026-10-04/server-lint.log](#): one error.

[.qa-audit/production-2026-10-04/client-lint.log](#): 30 errors, 20 warnings. Server workflow pulls main, npm install, PM2 restart/save without test/build/health gates.

What is happening / root cause: The active repositories do not have the ci.yml promised by their READMEs. Client one-off fix scripts are included in lint and account for the lint errors; the server has a genuine unused variable. An archived root workflow is not a gate for these separate repos.

Why it matters / impact: Security: deployment assurance weakness, not proof of an exploited deployment. Business/users: broken changes can reach releases unnoticed. Likelihood: high on normal push. Exploitability: not an attacker finding. Production: unreliable release process.

How to reproduce: Run npm run lint in each repo and inspect the active .github/workflows directories. Read the saved logs and server deploy workflow. No deployment was triggered.

Recommended fix / architecture: Remove/archive reviewed one-off scripts or configure their correct environment, fix genuine lint issues, then require lint/test/build before deployment. Use npm ci, a deploy health check and a rollback-capable release. Update docs to match actual workflows.

Priority: P1 - next remediation cycle.

Regression test required / verification: YES - intentional lint/test/build failure blocks deploy; successful staging deploy passes health check; rollback procedure is exercised separately.

D02 - Small dead exports and duplicate initialization remain

ID: D02

Severity: Low

Category: Dead Code / Drift

Status: CONFIRMED on the tested local snapshot.

Location / file / line: [client/src/lib/constants.js:171](#); [client/src/lib/utls.js:127](#); [client/src/main.jsx:30](#)

Component / API / data: Client constants, utility and entry point

Evidence: [.qa-audit/production-2026-10-04/dead-export-candidates.json](#);
[.qa-audit/production-2026-10-04/static-evidence.json](#). PAYROLL_STATUS_VARIANT and client lineAmount have no remaining AST code references in the scan; main.jsx awaits the same i18nReady promise twice.

What is happening / root cause: These are deletion/consolidation candidates. Most other zero-import exports are still used inside their module and must not have their implementation removed.

Why it matters / impact: Security/business: low maintainability cost. Users: no measured runtime slowdown from these small items. Likelihood: persistent code clutter. Exploitability: not applicable. Production: not a blocker.

How to reproduce: Search imports/identifier references across client/server/mcp source using static-audit.mjs and the dead-export appendix; inspect both await sites in main.jsx.

Recommended fix / architecture: Remove only truly unused declarations after confirming external/script consumers. Remove redundant export modifiers from locally used helpers only if desired. Keep one i18n initialization await. Do not pursue this before measured query/timeout fixes.

Priority: P2 - scheduled cleanup.

Regression test required / verification: NO new unit test for these reversible deletions; existing lint/build and an app boot smoke test are sufficient.

Section Access, role drift and IDOR coverage

All 38 server SECTION_KEYS matched the client module catalog in the static check; no invalid literal section key was found. Router aliases were resolved before evaluating guards. The outstanding confirmed client-only protection is outsourced-worker reads (S01). The deployment list/detail ownership mismatch is A01. The previous broad scan's raw "mutationsWithoutRouteGate" list includes aliases and is not itself a findings list.

Eleven mutating route declarations have none of requireRoles/requireStaff/requireSectionAccess after resolving aliases: login, refresh, logout, own password, own avatar PATCH/DELETE, public NFC click event, own notification read/read-all, and push subscribe/unsubscribe. These are intentional authentication/self-service/public-token surfaces rather than eleven automatic vulnerabilities. Their actual checks and all no-validate declarations appear in

[.qa-audit/production-2026-10-04/API-SECURITY-MATRIX.md](#). Parameterless routes need no schema, multipart upload validation is not ordinary JSON validation, and public NFC handlers use manual token/target checks. Outsourced raw query input is a confirmed gap. Wider schema coverage is listed without inventing a vulnerability for every absent validate().

The per-endpoint matrix includes static ownership markers; absence of a marker means **manual verification required**, not "no ownership check." Dynamic checks focused on employee/team, ESS documents/leave/advances, notifications, requirements, daily updates and deployment IDs. It does not certify every permissible custom Section Access grant. No additional HOLIDAY_MANAGE_ROLES-style mismatch was reproduced; role comparisons alone are not evidence that a documented role floor is wrong.

Secrets, injection, errors, dependencies and headers

No confirmed hardcoded production secret was found in the targeted source and reachable-history pattern scan. The only flagged credential URI was a placeholder in `server/.env.example` and its historical version. Actual server/client environment files have configured credentials/DSNs; values were neither copied into this report nor used for mutation tests. Public Vite API URLs and Sentry client DSNs are not automatically private credentials. This is not a guarantee against every possible secret format, ignored binary, external artifact or historic backup; it was not a full entropy-based scanner run.

Runtime-source scans did not find `eval/new Function`, child-process execution or dynamic require used as an application input execution path. No command or NoSQL operator injection was reproduced. The direct regex input failure is F08. Login object operators were rejected. No stored/reflected XSS exploit was established; sanitizer and public NFC paths still warrant dedicated browser payload coverage before calling that area fully tested.

Production error probes returned generic 500s without stacks/raw Mongoose internals. The error handler explicitly gates stacks by production mode and normalizes common Mongoose cases. Oversized JSON status handling is F09. Sensitive legacy Cloudinary delivery URLs were not inventoried in a real database; migration of historic public assets requires separate verification.

Saved npm audit results: [.qa-audit/production-2026-10-04/server-dependencies.json](#) and [.qa-audit/production-2026-10-04/client-dependencies.json](#). Server reported four package entries (one High, three Moderate); client twelve entries (eight High, four Moderate), all client entries marked dev-only. Counts are dependency graph entries, not independent remotely exploitable vulnerabilities. Server brace-expansion is transitive through archive tooling with no attacker-controlled pattern path demonstrated. The Multer advisory concerns `DiskStorage` orphaned writes; inspected document storage is memory/Cloudinary, so the advisory's specific disk-write path was not reproduced. UUID advisory applicability depends on the version/API used; no application exploit was demonstrated. Client build/Capacitor/sharp tooling still needs controlled updates, because build-time risks matter, but must not be mislabeled as eight production browser vulnerabilities. MCP dependencies were not separately scanned. No automatic force upgrades were made.

The development frontend response lacked CSP, frame protection, HSTS and Permissions-Policy in the observed response; `nosniff` and `Referrer-Policy` were present. The separate API had Helmet CSP, HSTS and frame protection. This is a hardening recommendation, not a reproduced clickjacking/XSS exploit. Roll out frontend CSP in report-only mode first, explicitly allowing the real API, Sentry and asset sources; consider frame-ancestors and a deployment-appropriate HSTS policy. A static HTML `Access-Control-Allow-Origin:*` is not the same as a credentialed API wildcard exposure. Compression and protocol performance cannot be inferred from a probe that did not negotiate every encoding.

Error tracking and operations

Error tracking **is implemented**: server Winston plus Sentry capture, frontend Sentry and error boundaries, and business audit events. Local server SENTRY_DSN and client VITE_SENTRY_DSN are set. Therefore “there is no error logging” would be incorrect. Delivery to your actual Sentry organization, alert recipients, release/source-map association, retention, uptime alerts and on-call response were not verified. The static dashboard health message is not evidence of monitoring working.

The smallest operational next step is to verify the existing setup: emit a sanitized staging test error from each layer, confirm the event and alert reach the intended owner, attach release identifiers, and set up an external health check. Configure retention/rotation for file logs if the host does not already manage it. Restore a backup into a separate database and record recovery time before asserting disaster-recovery readiness. Do not send real payroll or identity fields to test monitoring.

Failure review: bad inputs/unknown errors are mostly normalized; oversized JSON is misclassified. Authentication expiry has a refresh/retry path, but a hanging refresh has no timeout. Same-month hours deduplication worked; settlement deduplication did not. The deploy script has no demonstrated tested rollback. Real database outage, full server restart under writes, lost third-party responses, multi-instance limiter coordination and restore recovery were not injected or verified remotely.

Performance and UX priorities

1. **Batch target calculations (P02).** This is the strongest measured backend opportunity. Preserve financial math while eliminating repeated client-history reads.
2. **Fix list bounds (S01), IP contention (P01) and timeouts (P03).** These affect payload size, failed page requests and apparent hanging. Do not claim a production p95 until measured there.
3. **Measure before further refactoring.** Capture slow-route duration/query count, payload sizes and frontend interaction traces on representative sanitized data. The outsourced name query already used an index (100 documents / 101 keys examined); adding another index blindly is not justified.
4. **Keep code splitting.** Build already creates lazy feature chunks. Largest observed react-dom chunk was about 279 KB raw / 85.5 KB gzip; Arabic and schema chunks about 123.5/126 KB raw. These sizes alone do not prove the UI bottleneck. No Lighthouse/Core Web Vitals certification was performed.
5. **Improve error states.** A missing payslip API should not ask users to check their connection. Show actionable permission/unsupported/timeout states; retain entered form data after transient failures; give mutation retries idempotency protection.
6. **Accessible dialogs and zoom.** Fix U01. client/index.html:5 requests maximum-scale=1 and user-scalable=0; remove this restriction and verify 200% zoom and Android WebView behavior. Browser support varies, so this is not a claim that all browsers currently block zoom.
7. **Avoid broad rewrites.** Extract repeated table-preference effects only after tests, and share validation fixtures across frontend/backend. Do not remove server validation because the client duplicates it.

Dead code, duplicate logic and documentation handoff

See [.qa-audit/production-2026-10-04/DEAD-CODE-AND-DUPLICATION.md](#) for exact locations and the full candidate list. The scan detected repeated IBAN validation, mobilisation refinements, table-preference effects, outsourced create/update refinements and small hub wrappers. Not all are defects: client/server validation is intentional defense at two boundaries, and short wrappers have negligible runtime cost. Prefer shared cases/contracts to a risky package/repository reorganization.

Retain migrations and feature-history docs until their operational purpose is checked. Review tracked `fix_mob*.cjs` / `fix_swap*.cjs` / `fix_widths.cjs` and `server/test_db.js` as one-off maintenance candidates; do not blanket-delete them. Archive old audit/history documents behind an index with dates rather than leave contradictory current setup instructions. Server README:75 and client README:63 claim `ci.yml` that is absent in their active repos. The limiter comments overstate the solved shared-office problem (P01). One plain-language architecture explanation is provided above; replacing otherwise useful READMEs is unnecessary.

Scores and production checklist

Scores are provisional evidence-based handoff ratings, not compliance certification or an arithmetic release gate. Limited-access categories are capped rather than assumed healthy.

Category	Score / 10	Status	Basis
Architecture	7	PARTIAL	Clear client/service/model separation and shared guards; finance/report services repeat work.
Frontend	6	PARTIAL	Build and sampled pages work; absent payslip API and unbounded waits.
Backend	5	FAIL	Passing existing suite, but PATCH data loss and invalid settlement persistence.
API	5	FAIL	Broad schema/guard coverage; outsourced disclosure and targeted 500s.
Database	5	FAIL	Useful indexes and duplicate-hour protection; missing final-settlement invariant. Backup restore NOT TESTED.
Authentication	7	PARTIAL	Refresh rotation, token version and origin controls; forced-change bypass. Recovery/MFA not fully tested.
Authorization	4	FAIL	Several strong ownership checks; S01 and inconsistent coordinator policy.
Security	4	FAIL	One confirmed High confidentiality flaw; no critical exploit found.
UI	7	PARTIAL	Consistent sampled hubs/forms; full device/page matrix NOT TESTED.
UX	5	FAIL	Working primary flows; nonfunctional payslips and misleading health/error states.
Accessibility	4	FAIL	Shared modal keyboard failure; zoom restriction. Full WCAG/contrast audit NOT TESTED.
Performance	5	PARTIAL	Lazy chunks and indexed search; 234-query report and shared-IP contention.
SEO	6	PARTIAL	Internal CRM has limited SEO value; public NFC has metadata. Public crawl/CWV NOT TESTED.
Testing	4	PARTIAL	34 backend tests pass; no client automated test script; targeted audit finds uncovered regressions.
DevOps / infrastructure	4	PARTIAL	Build/deploy mechanism exists; missing quality gates, rollback and host settings unverified.
Observability / logging	6	PARTIAL	Winston/Sentry/audit exist; alert delivery and retention NOT TESTED; static health card.
Maintainability	5	PARTIAL	Modular code and real history; failed lint, patch scripts and redundant exports.
Documentation	6	PARTIAL	READMEs and architecture history exist; CI/deployment drift needs correction.

Backups and disaster recovery: **NOT TESTED**. No restore certificate or cloud policy was available. Mobile APK release readiness: **NOT TESTED**; Capacitor Android project is in client/android (app ID com.aljazeera.crm, version 1.1.2 / code 4 in inspected configuration). Signing credentials were not exposed. No installable APK is being certified by this web/API audit.

Remediation roadmap and release gate

Phase 0 - release blockers: S01, F01, F02. Fix authorization, preserve omitted metadata, reject invalid computed money. Add their regression tests, inspect whether deployed data needs reconciliation, and verify the exact deployed revision in staging.

Phase 1 - security hardening: S02; bound outsourced queries; align download policies; review legacy public files; triage reachable dependency advisories; stage CSP/frame policy changes. Validate each fix without broad permission grants.

Phase 2 - reliability: F03-F09, A01, D01 and O01. Enforce final-settlement uniqueness compatible with simulations/rehiring; repair leave invariants/date windows; resolve payslip product/API contract; add CI gates; verify alerts, rollback and backup restore. Resolve EOSB business policy before financial acceptance.

Phase 3 - UX/accessibility: U01, differentiated errors, zoom, mobile/tablet/desktop and Arabic/RTL regression passes. Add browser tests for login, employee edit, leave approval and payslip visibility.

Phase 4 - performance: P02 batching first, then P01 budget sizing and P03 timeouts/cancellation. Measure before/after query counts and staging p95; retain correctness checks for finance.

Phase 5 - code quality: D02, small shared validation/effect helpers, reviewed removal/archive of maintenance scripts, and README/CI reconciliation. No architectural rewrite required.

Phase 6 - longer term: realistic workload/performance budgets, owner-aware API integration tests for all configurable grants, deliberate MFA/recovery policy, dependency automation, and independently tested Android release pipeline.

Unverified items and decisions needed

- **EOSB policy:** current helper returns a flat two months' wage for both a two-year contract completion and a ten-year termination in the fixture, with 45 leave days after the threshold. Whether this is intended company policy is unanswered. This is a verified calculation but **not classified as a legal/policy bug without your rule**. CurrentEmployee simulation was added during audit; its full latest workflow needs acceptance tests.
- **Deployment identity:** link the released frontend/API commits to this report before treating local findings as remotely reproduced. Only public deployment smoke checks were run.
- **Monitoring / recovery:** verify alert destination, source maps, cloud logs, rotation, database access roles, encrypted transport/storage policies, backup schedule and an actual restore.
- **Potential push-endpoint SSRF:** endpoint URL acceptance deserves allowlist/network-boundary review; no outbound private-network request was attempted and no exploit is claimed.
- **Full journey/device coverage:** payroll calculation/finalization and employee payslip delivery, all upload/download variants, all custom permission combinations, full Arabic/mobile/tablet/desktop accessibility and physical Android install are not fully verified.

The missing operator access does not prevent reporting the reproduced release blockers, but it prevents a GO decision. **Security: FAIL. Confirmed critical vulnerabilities: 0. Confirmed High security vulnerabilities: 1. Confirmed Medium security vulnerabilities: 1. High-severity production blockers: 3. Overall risk: HIGH. Recommendation: NO-GO until blockers and the release verification gate are closed.**

Evidence and reproduction guide

The supplied scripts are audit probes, not an installed product test suite. Use a fresh isolated database: start `local-runtime.mjs`, then `http-tests.mjs` → `workflows.mjs` → `deep-tests.mjs` → `latest-regressions.mjs` → `scoping-performance.mjs` → `final-probes.mjs` → `input-probes.mjs` → `billing-flow.mjs`. They intentionally create invalid/duplicate synthetic records, so **never point them at a real database**. They override configuration through `audit-env.mjs` and use an installed MongoDB binary. Some depend on earlier fixture IDs and October 2026 dates; reruns at another date need date fixtures reviewed. `STOP-ISOLATED-RUNTIME` is the controlled shutdown marker; remove that marker only when intentionally starting a fresh audit run. Keep the raw JSON/logs alongside the report for exact request/results.

[.qa-audit/production-2026-10-04/API-INVENTORY.md](#) - all routes, guards, schemas and static data access.

[.qa-audit/production-2026-10-04/API-SECURITY-MATRIX.md](#) - authentication/ownership markers, validation gaps and runtime scope.

[.qa-audit/production-2026-10-04/DEAD-CODE-AND-DUPLICATION.md](#) - deletion candidates versus locally used exports and intentional duplication.

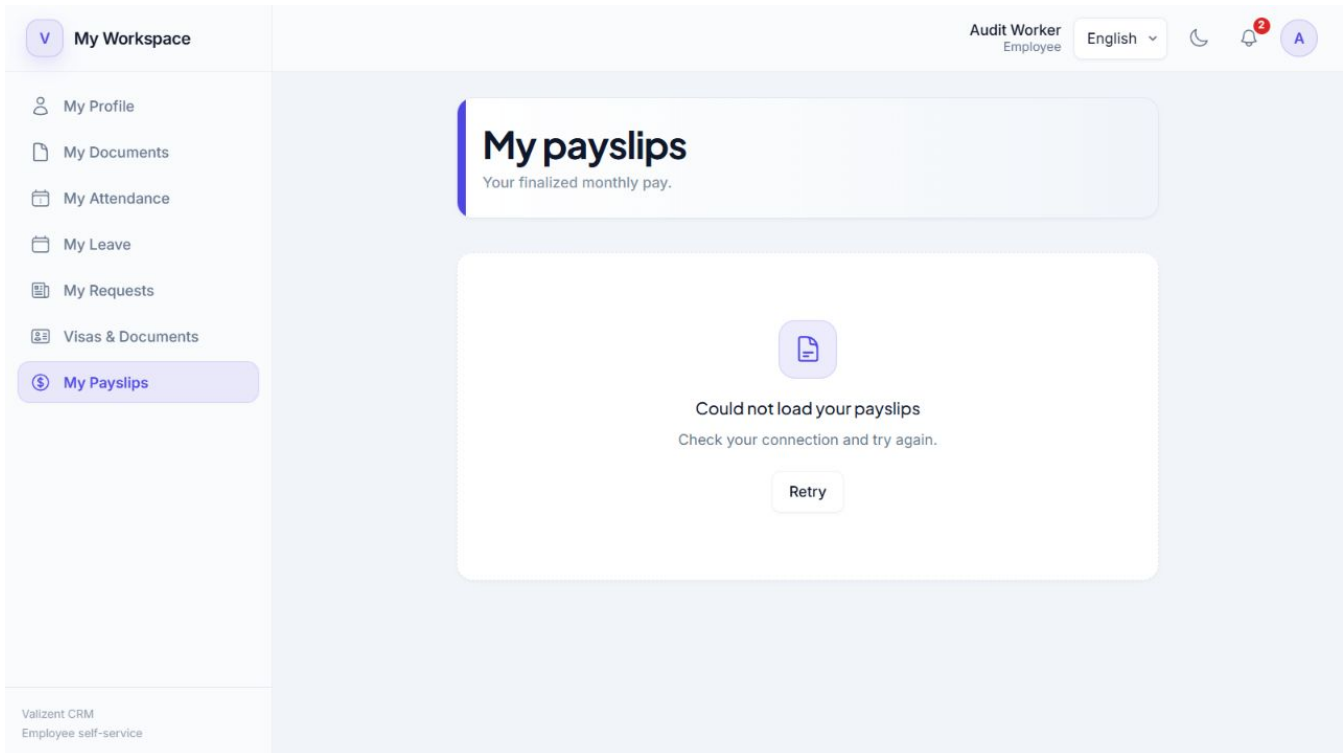
[.qa-audit/production-2026-10-04/FINDINGS.json](#) - structured confirmed findings.

Risk taxonomy reference: the [OWASP Top 10:2025](#) distinguishes access control, misconfiguration, supply-chain, authentication, design/integrity, logging and exceptional-condition risks. This report is not an OWASP certification and does not equate an untested category with a pass.

Cleanup and visual evidence

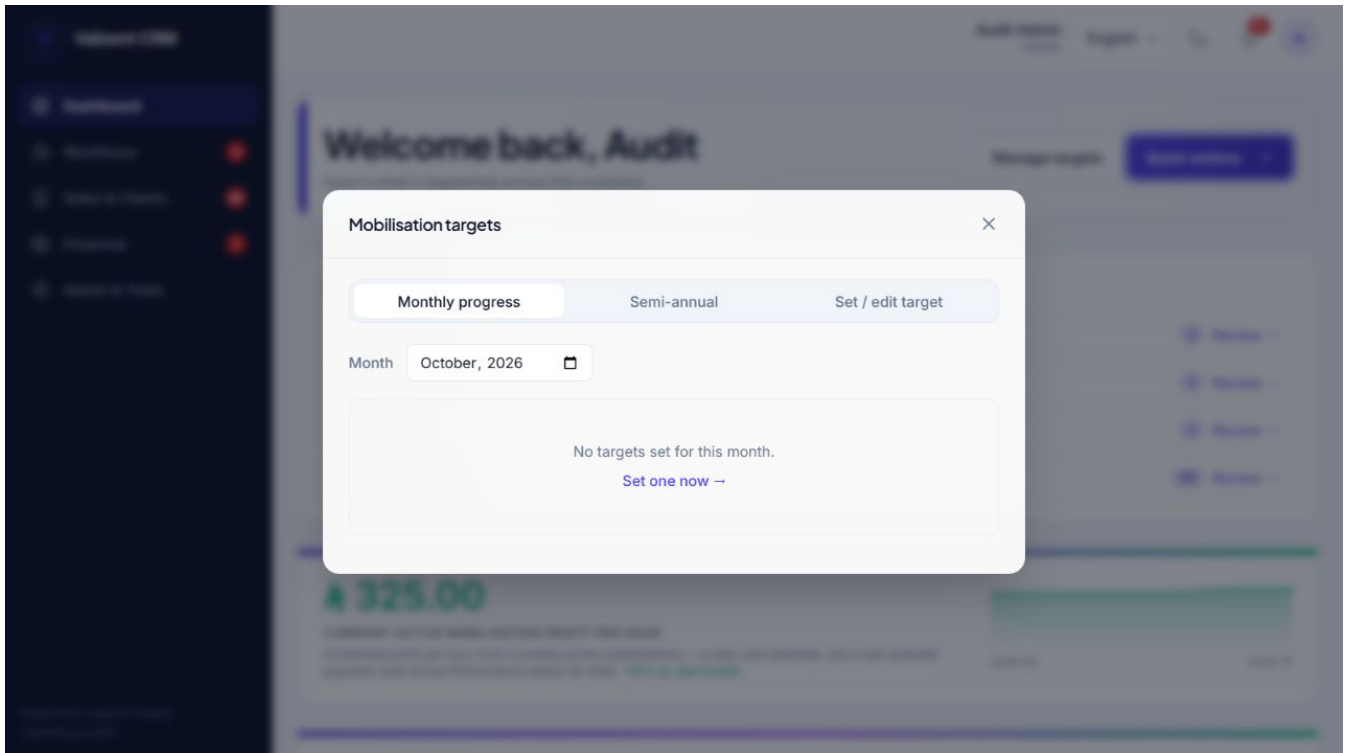
The isolated API and MongoDB replica set were stopped through the audit launcher; it confirmed disposal of the temporary database. The audit Vite process was stopped and its temporary browser tab closed. All ten synthetic accounts and synthetic business records existed only in that disposable database. The audit ports no longer respond; saved evidence remains in this directory. No remote test accounts were created.

Worker payslip failure captured in the local browser:



Worker payslip API failure

Target modal used for the keyboard-focus reproduction (focus details are in modal-focus.json):



Target dialog with background focus

Appendix - architecture diagram source

Original Mermaid source, preserved for editing and exact reference.

flowchart TD

```
A[Browser / Android WebView: untrusted client] --> B[Cloudflare Pages frontend]
B --> C[HTTPS API boundary]
M[MCP client with user credentials] --> C
C --> D[Origin checks / rate limits / JWT authentication]
D --> E[Role + Section Access + ownership + Zod validation]
E --> F[Controllers and business services]
F --> G[(MongoDB: users, employees, approvals, finance)]
F --> H[Private uploads / Cloudinary delivery boundary]
F --> I[Push and external integrations]
F --> J[Winston / Sentry / audit events]
K[Public NFC token pages and click beacons] --> F
```

The exact original AUDIT-REPORT.md is embedded in this PDF, including original links, punctuation and Markdown formatting.

Original source SHA-256: 980ccdda77425c2ab41487c6d1ea4ce8659e59d704583512943e6493a0ee1e64